TECHNICAL ASSESSMENT - DATA ENGINEERING INTERVIEW

FAMILY BANK

SECTION A: SQL (10 marks)

Question 1

Write an SQL query to return all customers from Nairobi.

sql

```sql
-- Solution using basic WHERE clause
SELECT CustomerID, Name, City
FROM Customers
WHERE City = 'Nairobi';

-- Alternative solution with case-insensitive comparison
-- (handles potential data quality issues)
SELECT CustomerID, Name, City
FROM Customers
WHERE UPPER(TRIM(City)) = 'NAIROBI';

-- Comprehensive solution with NULL handling and indexing consideration
SELECT CustomerID, Name, City
FROM Customers
WHERE City = 'Nairobi'
  AND City IS NOT NULL;  -- Explicitly handle NULLs for better performance
```

Explanation: The query filters records where the City column equals 'Nairobi'. The first solution is the simplest and most efficient for exact matches. The alternative solutions demonstrate robust practices for handling variations in data quality, such as case sensitivity and leading/trailing spaces, which are common in real-world production databases.

Question 2

Write an SQL query that returns:

Name	TotalAmount
John	8000
Mary	7000

sql

```sql
-- Primary solution using INNER JOIN and GROUP BY
SELECT
    c.Name,
    SUM(o.Amount) AS TotalAmount
FROM Customers c
INNER JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.Name
ORDER BY TotalAmount DESC;  -- Optional: Sort by highest total first

-- Alternative solution that handles customers with no orders
SELECT
    c.Name,
    COALESCE(SUM(o.Amount), 0) AS TotalAmount  -- Shows 0 for customers without order
s
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.Name
ORDER BY TotalAmount DESC;

-- Advanced solution using Common Table Expression (CTE) for clarity
WITH CustomerTotals AS (
    SELECT
        c.CustomerID,
        c.Name,
        SUM(o.Amount) AS TotalAmount
    FROM Customers c
```

```
    INNER JOIN Orders o ON c.CustomerID = o.CustomerID
    GROUP BY c.CustomerID, c.Name
)
SELECT Name, TotalAmount
FROM CustomerTotals
ORDER BY TotalAmount DESC;
```

Explanation: The query joins the Customers and Orders tables on CustomerID, groups results by customer, and calculates the sum of amounts. The INNER JOIN ensures we only see customers who have placed orders. The alternative LEFT JOIN version includes customers with no orders (showing 0), which is more comprehensive for reporting scenarios.

Question 3

What is the difference between INNER JOIN and LEFT JOIN?

INNER JOIN:

- Returns only the rows where there is a match between both tables based on the join condition
- Excludes records from either table that don't have a matching record in the other table
- If a customer has no orders, they won't appear in the result set
- Most efficient join type as it processes only matched records

LEFT JOIN (or LEFT OUTER JOIN):

- Returns ALL rows from the left (first) table, regardless of whether there's a match in the right table
- For unmatched rows in the right table, NULL values are returned for right table columns
- If a customer has no orders, the customer still appears with NULL order columns
- Used when you need to see all records from the primary entity, even if they lack related data

Key Differences:

Aspect	INNER JOIN	LEFT JOIN
Return Records	Only matched records	All records from left table + matched from right
Unmatched Records	Excluded entirely	Included with NULL values for right table

Aspect	INNER JOIN	LEFT JOIN
Use Case	When you only need records with complete data	When you need all records from the primary table, those without relationships
Performance	Generally faster	Can be slower due to additional NULL handling
Data Completeness	Guarantees both sides have data	May introduce NULL values requiring handling

Example Scenario:

```sql
-- Using the Customers and Orders tables:
-- INNER JOIN: Shows only customers with orders
SELECT c.Name, o.Amount
FROM Customers c
INNER JOIN Orders o ON c.CustomerID = o.CustomerID;
-- Result: John (5000), John (3000), Mary (7000)
-- Peter (with no orders) is excluded

-- LEFT JOIN: Shows all customers, even those without orders
SELECT c.Name, o.Amount
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID;
-- Result: John (5000), John (3000), Mary (7000), Peter (NULL)
-- Peter appears with NULL for order amount
```

SECTION B: Data Engineering Concepts (10 marks)

Question 4

What is ETL? Explain each stage.

ETL (Extract, Transform, Load) is a data integration process that extracts data from various sources, transforms it to fit operational needs, and loads it into a target database or data warehouse. It's the backbone of modern data engineering.

1. EXTRACT (Extraction)

Objective: Pull raw data from multiple source systems.

Key Activities:

- **Identify sources:** Relational databases (PostgreSQL, MySQL, Oracle), NoSQL databases (MongoDB, Cassandra), flat files (CSV, JSON, XML), APIs (REST, SOAP), streaming data (Kafka, Kinesis), and legacy systems
- **Extraction methods:**
  - *Full extract:* Complete dump of all data (used for initial loads)
  - *Incremental extract:* Only new or changed data since last extraction (uses timestamps, CDC - Change Data Capture)
  - *Delta extraction:* Extract based on change logs
- **Challenges:**
  - Handling large volumes without impacting source systems
  - Dealing with system downtime and network issues
  - Maintaining data integrity during extraction
  - Handling various data formats and structures

2. TRANSFORM (Transformation)

Objective: Clean, validate, and transform raw data into a usable format.

Key Activities:

- **Data Cleaning:**
  - Handle NULL/missing values (imputation, deletion, or default values)
  - Remove duplicates and outliers
  - Standardize data formats (dates, currencies, units)
  - Correct data entry errors
- **Data Enrichment:**
  - Join/merge multiple datasets
  - Add calculated fields (age from birthdate, tax from sales)
  - Lookup reference data (geocoding, product categories)
- **Data Validation:**
  - Check business rules and constraints
  - Validate data types and formats
  - Ensure referential integrity
- **Data Normalization:**
  - Scale numerical data

- Standardize categorical values
- Apply business logic rules
- **Data Restructuring:**
- Pivot/unpivot tables
- Split/combine columns
- Aggregate data for summary views

Advanced Transformations:

- **Sensitivity Masking:** Anonymize PII (Personally Identifiable Information)
- **Data Deduplication:** Implement sophisticated matching algorithms
- **Rule-based Logic:** Apply complex business rules
- **Machine Learning Integration:** Apply predictive models to enrich data

3. LOAD (Loading)

Objective: Load transformed data into the target system.

Key Activities:

- **Loading Strategies:**
- *Full Load:* Replace entire target table (for initial or small data)
- *Incremental Load:* Append only new data
- *Upsert (MERGE):* Insert new records, update existing ones
- *Type 1 SCD:* Overwrite historical data
- *Type 2 SCD:* Maintain history with versioning
- **Performance Optimization:**
- Batch loading for large datasets
- Using bulk copy operations
- Partitioning large tables
- Creating appropriate indexes after load
- **Error Handling:**
- Implement retry mechanisms for failed loads
- Log errors for investigation
- Maintain a dead letter queue for problematic records
- **Data Quality Checks:**
- Validate row counts and totals
- Verify data integrity and referential constraints
- Run reconciliation queries

Modern ETL vs ELT:

- **ETL (Traditional):** Transform before loading (uses separate transformation server)
- **ELT (Modern):** Extract, Load, Transform (transformations performed within the data warehouse using its processing power, leveraging cloud scalability)

Question 5

What is a Data Warehouse? Why do organizations use one?

Definition: A Data Warehouse (DWH) is a centralized repository designed specifically for storing, managing, and analyzing structured data from multiple sources. It's optimized for reporting and analytical purposes rather than transactional processing.

Key Characteristics:

- **Subject-Oriented:** Organized around major business subjects (customers, products, sales, finances)
- **Integrated:** Data from disparate sources is standardized (consistent naming, formats, codes)
- **Time-Variant:** Stores historical data over long periods (often 5-10 years)
- **Non-Volatile:** Data is read-only and stable once loaded; updates are batch-based

Why Organizations Use Data Warehouses:

1. Strategic Decision Making:

- Provides a "single source of truth" for enterprise-wide reporting
- Enables data-driven decision making across all levels
- Supports complex analytical queries without impacting operational systems

2. Data Integration:

- Consolidates data from disparate systems (ERP, CRM, HR, POS, etc.)
- Resolves data inconsistencies and conflicts between systems
- Creates a unified view of the business

3. Performance Optimization:

- Optimized for read-heavy analytical workloads
- Separates analytical workloads from transactional systems
- Implements specialized indexing and storage structures

4. Historical Analysis:

- Maintains historical data for trend analysis
- Enables time-series analysis and year-over-year comparisons
- Supports data mining and predictive analytics

5. Data Quality:

- Standardizes, cleanses, and validates data
- Enforces business rules and data governance
- Provides consistent definitions and metrics across the organization

6. Compliance and Security:

- Centralized security and access controls
- Audit trail for data lineage and changes
- Supports regulatory compliance (GDPR, Basel, IFRS)

7. Enhanced Business Intelligence:

- Powers dashboards, visualizations, and reports
- Enables ad-hoc querying by business analysts
- Supports complex business metrics and KPIs

8. Scalability:

- Modern cloud data warehouses (Snowflake, Redshift, BigQuery) scale elastically
- Separates storage and compute for cost efficiency
- Supports growing data volumes and user bases

Question 6

What is Apache Airflow used for?

Apache Airflow is an open-source platform for programmatically authoring, scheduling, and monitoring workflows (Directed Acyclic Graphs - DAGs) of data pipelines and data orchestration.

Primary Uses:

1. Workflow Orchestration:

- Schedule and orchestrate complex data pipelines
- Manage dependencies between tasks
- Define workflows as Python code (DAGs)
- Control execution order of tasks

2. Data Pipeline Management:

- ETL/ELT pipeline automation
- Data ingestion from various sources
- Data transformation and loading orchestration

- Data quality and validation tasks

3. Monitoring and Alerting:

- Track task and DAG execution status
- Set up failure alerts and notifications
- Visualize pipeline dependencies and execution history
- Monitor SLAs (Service Level Agreements)

4. Error Handling and Retries:

- Implement retry mechanisms for failed tasks
- Configure backoff strategies for API rate limits
- Handle partial failures gracefully
- Automate recovery procedures

5. Scheduling:

- Cron-based scheduling (hourly, daily, weekly)
- Trigger-based execution (file arrival, API webhook)
- Event-driven workflows
- Backfilling (re-running historical data)

6. Version Control and CI/CD:

- Workflow definitions as code in Git
- Track changes and rollback capabilities
- Test workflows before deployment
- Integrate with development environments

Key Components:

- **Scheduler:** Triggers scheduled workflows
- **Executor:** Executes task instances
- **Webserver:** Provides UI for monitoring and management
- **Metadata Database:** Stores state of DAGs and tasks
- **Worker:** Runs tasks (in distributed mode)

Common Use Cases in Banking/Finance:

- Daily ETL pipeline for transaction data
- Regulatory reporting workflows
- Fraud detection model retraining
- Customer data synchronization
- Risk calculation and reporting

Question 7

A data pipeline loads customer transactions every night. One morning the pipeline fails. What are the first three things you would check?

First Three Checks:

1. Check Pipeline Execution Logs and Error Messages

- Immediately review Airflow/DAG execution logs or pipeline job logs
- Identify specific error type: connection failure, timeout, data format issue, permission error, or business rule violation
- Check timestamps to determine exact failure point (extract, transform, or load stage)
- Look for stack traces or specific error codes for debugging

2. Verify Source System and Data Availability

- Check if the source database/system is online and accessible
- Verify network connectivity between pipeline and source
- Confirm that source data files exist and are complete
- Check for schema changes in source (new columns, changed data types)
- Validate source system logs for any issues during the extract window
- Check if there was a source system maintenance or update overnight

3. Validate Target System Status and Data Integrity

- Check if the data warehouse/target database is operational
- Verify disk space availability (no storage full issues)
- Check for any locks or blocking queries on target tables
- Validate that target tables exist with expected schema
- Review target system logs for any errors during load attempt
- Check if there were any permission changes affecting write operations

Subsequent Checks (if initial checks don't reveal the issue):

- **Data Volume Checks:** Unexpected surge in data volume causing memory/timeouts
- **Data Quality Issues:** Invalid data formats, NULL constraints, duplicate primary keys
- **Resource Constraints:** CPU, memory, or connection pool exhaustion
- **Dependency Failures:** Previous tasks or external systems unavailable
- **Configuration Changes:** Environment variables, connection strings, or credentials expired

Post-Mortem Actions:

- Implement enhanced monitoring and alerting

- Add data validation checks before loading
- Create retry logic with exponential backoff
- Set up automated recovery mechanisms
- Document root cause and preventive measures

SECTION C: Python (10 marks)

Question 8

What will the following code output?

python

```python
numbers = [2, 4, 6, 8]
print(sum(numbers))
```

Output:

text

```
20
```

Explanation: The `sum()` function takes an iterable (list, tuple, etc.) and returns the arithmetic sum of all elements. In this case: 2 + 4 + 6 + 8 = 20.

Advanced Consideration: If we were to write our own sum function for educational purposes:

python

```python
def custom_sum(iterable):
    total = 0
    for num in iterable:
        total += num
    return total

# Alternative using reduce (functional programming)
from functools import reduce
print(reduce(lambda x, y: x + y, numbers))  # Also returns 20
```

Question 9

Write a Python function that returns the average of a list of numbers.

python

```python
def average(numbers):
    """
    Calculate the average (mean) of a list of numbers.

    Args:
        numbers (list): List of numeric values

    Returns:
        float: The arithmetic mean of the numbers

    Raises:
        ValueError: If the list is empty
    """
    if not numbers:  # Check for empty list
        raise ValueError("Cannot compute average of empty list")

    return sum(numbers) / len(numbers)

# Example usage:
print(average([2, 4, 6]))  # Output: 4.0

# Advanced version with better error handling and data validation
def average_advanced(numbers, ignore_none=False, ignore_strings=True):
    """
    Advanced average function with input validation.

    Args:
        numbers (list): List of numeric values
        ignore_none (bool): Whether to ignore None values
        ignore_strings (bool): Whether to ignore string values

    Returns:
        float: The arithmetic mean
    """
```

```python
    if ignore_strings:
        numbers = [x for x in numbers if not isinstance(x, (str, bytes))]

    if ignore_none:
        numbers = [x for x in numbers if x is not None]

    if not numbers:
        raise ValueError("No valid numbers to compute average")

    # Ensure all values are numeric
    numeric_values = []
    for x in numbers:
        try:
            numeric_values.append(float(x))
        except (TypeError, ValueError):
            continue

    if not numeric_values:
        raise ValueError("No numeric values found")

    return sum(numeric_values) / len(numeric_values)

# Alternative implementation using statistics module (Python 3.4+)
import statistics
def average_statistics(numbers):
    try:
        return statistics.mean(numbers)
    except statistics.StatisticsError:
        raise ValueError("Cannot compute average of empty list")
```

Question 10

Write code to create a new column called Tax equal to 16% of Sales.

python

```python
import pandas as pd
```

```python
df = pd.DataFrame({
    'Sales': [100, 200, 300]
})

# Solution 1: Direct column assignment (most common)
df['Tax'] = df['Sales'] * 0.16

# Solution 2: Using .assign() method (for method chaining)
df = df.assign(Tax=df['Sales'] * 0.16)

# Solution 3: Using apply() (more flexible for complex calculations)
df['Tax'] = df['Sales'].apply(lambda x: x * 0.16)

# Solution 4: Using numpy for vectorized operations
import numpy as np
df['Tax'] = np.multiply(df['Sales'], 0.16)

# Advanced: Round to 2 decimal places (monetary format)
df['Tax'] = (df['Sales'] * 0.16).round(2)

# Advanced: Include GST/VAT calculation with different categories
def calculate_tax(sales, tax_rate=0.16):
    """Calculate tax with different rates for different categories"""
    return sales * tax_rate

df['Tax'] = df['Sales'].apply(calculate_tax)

# Resulting DataFrame:
#    Sales  Tax
# 0    100  16.0
# 1    200  32.0
# 2    300  48.0

# Display the result
print(df)
```

SECTION D: Scenario Questions (10 marks)

Question 11

Family Bank wants a dashboard showing:

- Daily transactions
- Monthly transactions
- Branch performance

Describe a high-level data pipeline that would support this.

High-Level Data Pipeline Architecture:

1. Data Sources & Ingestion Layer:

- **Source Systems:**
  - Core Banking System (T24/Bancs) - Transaction records, account information
  - Branch Database - Branch details, employee data
  - ATM/POS Networks - Digital transaction logs
  - Mobile Banking API - Mobile transaction data
  - CRM System - Customer information
- **Ingestion Methods:**
  - CDC (Change Data Capture) from banking system using Debezium
  - API integrations for real-time mobile transactions
  - Batch ETL jobs for end-of-day settlement data
  - File ingestion (CSV/Parquet) for historical data

2. Data Staging & Storage Layer:

- **Bronze Layer (Raw Data):**
  - Store raw, unprocessed data in data lake (S3/ADLS)
  - Maintain data as-is for audit and reprocessing
  - Partition by date for efficient querying
- **Silver Layer (Cleaned Data):**
  - Standardized and validated data
  - Business rules applied (transaction categorization)
  - Data quality checks passed
- **Gold Layer (Aggregated Data):**
  - Star schema optimized for dashboarding
  - Fact tables: Transactions, Daily Balances
  - Dimension tables: Customer, Branch, Date, Product

3. Data Processing & Transformation Layer:

- **Tools:** Apache Spark, Dataflow, or dbt
- **Key Transformations:**
  - **Daily Aggregation:**

```
sql

SELECT
    transaction_date,
    branch_id,
    COUNT(*) as daily_transaction_count,
    SUM(amount) as daily_volume
FROM transactions
GROUP BY transaction_date, branch_id
```

  - **Monthly Aggregation:**

```
sql

SELECT
    DATE_TRUNC('month', transaction_date) as month,
    branch_id,
    COUNT(*) as monthly_transactions,
    SUM(amount) as monthly_volume,
    AVG(amount) as avg_transaction_value
FROM transactions
GROUP BY DATE_TRUNC('month', transaction_date), branch_id
```

  - **Branch Performance Metrics:**

```
sql

SELECT
    branch_id,
    branch_name,
    region,
    transaction_volume,
    customer_acquisition_rate,
    transaction_growth_mom,
    performance_ranking
FROM branch_performance_view
```

4. Data Orchestration & Scheduling:

- **Apache Airflow DAG Structure:**

```
text

Task 1: Extract from Core Banking (11:00 PM)
  ↓
Task 2: Extract from Mobile Banking (11:30 PM)
  ↓
Task 3: Data Validation & Cleaning (12:00 AM)
  ↓
Task 4: Daily Aggregation (12:30 AM)
  ↓
Task 5: Monthly Update (1st of month only)
  ↓
Task 6: Load to Data Warehouse (1:00 AM)
  ↓
Task 7: Refresh Dashboard Views (1:30 AM)
  ↓
Task 8: Send Success/Failure Notifications
```

5. Data Serving & Visualization:

- **Data Warehouse:** Snowflake/BigQuery/Redshift
- **Dashboard Tool:** Power BI/Tableau/Looker
- **Dashboard Views:**
  - **Daily Dashboard:**
    - Today's transaction volume vs. yesterday
    - Real-time transaction monitor
    - Top branches by transaction count
    - Unusual transaction patterns alerts
  - **Monthly Dashboard:**
    - Month-over-month growth trends
    - Branch ranking and performance
    - Product-wise transaction distribution
    - Customer segment analysis
  - **Branch Performance:**
    - Transaction volume heatmap
    - Branch efficiency ratios
    - Customer satisfaction scores
    - Staff productivity metrics

6. Data Quality & Monitoring:

- **Checkpoints:**
  - Data volume verification (expected vs. actual)
  - Null ratio checks in critical fields
  - Referential integrity validation
  - Data freshness monitoring
- **Alerting:**
  - Pipeline failure notifications
  - Data anomalies detection
  - SLA breach warnings
  - Dashboard data staleness alerts

7. Security & Compliance:

- Data encryption at rest and in transit
- Role-based access control (RBAC)
- Audit logging for all data access
- GDPR compliance for customer data
- Data retention policies (7-year requirement)

8. Scaling & Performance:

- **Scaling Strategy:**
  - Auto-scaling compute resources during peak load
  - Partition pruning for faster queries
  - Materialized views for frequent aggregations
  - Caching layer for dashboard performance

Question 12

A machine learning model for fraud detection has been developed. The Data Engineering team has been asked to deploy it. List four responsibilities the Data Engineer would have during deployment.

1. Model Serving Infrastructure and API Development

Responsibilities:

- Design and implement the model serving architecture (e.g., using TensorFlow Serving, SageMaker, or custom Flask/FastAPI endpoints)
- Create RESTful APIs or gRPC endpoints for real-time inference

- Configure load balancing and auto-scaling for production traffic
- Set up model versioning and A/B testing capabilities
- Implement request/response logging for audit purposes
- Establish metrics monitoring (latency, throughput, error rates)

Implementation Example:

python

```python
# Model serving API with FastAPI
from fastapi import FastAPI, HTTPException
import pandas as pd
import joblib
from pydantic import BaseModel

app = FastAPI()
model = joblib.load('fraud_model.pkl')

class TransactionRequest(BaseModel):
    transaction_id: str
    amount: float
    merchant: str
    location: str
    customer_id: str
    # ... other features

@app.post("/predict")
async def predict_fraud(request: TransactionRequest):
    try:
        # Transform request to model features
        features = prepare_features(request)
        prediction = model.predict(features)
        confidence = model.predict_proba(features)

        # Log request for audit
        log_prediction(request, prediction, confidence)

        return {
            "transaction_id": request.transaction_id,
            "is_fraud": bool(prediction[0]),
            "confidence": float(confidence.max()),
```

```
            "timestamp": datetime.now()
        }
    except Exception as e:
        # Log error and return appropriate response
        raise HTTPException(status_code=500, detail=str(e))
```

2. Data Pipeline Engineering for Feature Processing

Responsibilities:

- Build robust feature engineering pipelines that mirror training-time transformations
- Implement feature store integration for real-time and batch features
- Create data validation checks to ensure incoming data matches training schema
- Handle missing values, outliers, and data type conversions
- Implement data versioning and lineage tracking
- Ensure feature freshness (real-time vs. batch features)

Implementation Considerations:

python

```
# Feature transformation pipeline
class FraudFeatureTransformer:
    def __init__(self, feature_config):
        self.config = feature_config
        self.scaler = joblib.load('scaler.pkl')
        self.encoder = joblib.load('encoder.pkl')

    def transform(self, raw_data):
        # Validate input data
        self.validate_schema(raw_data)

        # Apply transformations
        features = {
            'amount_log': np.log1p(raw_data['amount']),
            'merchant_encoded': self.encoder.transform(raw_data['merchant']),
            'location_distance': self.calculate_distance(raw_data),
            'time_features': self.extract_time_features(raw_data),
            'avg_transaction_amount': self.get_feature_from_store(
                raw_data['customer_id'], 'avg_transaction_amount'
            ),
            'velocity_24h': self.calculate_transaction_velocity(raw_data)
        }
```

```
        return self.scaler.transform(features)
```

3. CI/CD Pipeline and Infrastructure Automation

Responsibilities:

- Implement automated deployment pipeline for model updates
- Configure environment isolation (dev, staging, production)
- Set up automated testing for model performance and accuracy
- Manage environment variables, secrets, and configuration
- Create rollback mechanisms for failed deployments
- Implement blue-green or canary deployment strategies

CI/CD Pipeline Components:

yaml

```yaml
# Example GitHub Actions workflow
name: Deploy Fraud Detection Model

on:
  push:
    branches: [main]
    paths:
      - 'models/fraud_model/**'
      - 'src/fraud_detection/**'

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Run unit tests
        run: pytest tests/
      - name: Validate model performance
        run: python tests/validate_model.py
      - name: Security scan
        run: bandit -r src/fraud_detection/

  deploy-staging:
    needs: test
    runs-on: ubuntu-latest
```

```
    steps:
      - name: Deploy to staging
        run: |
          kubectl set image deployment/fraud-detection \
            fraud-container=${{ env.REGISTRY }}/fraud-model:${{ github.sha }}
          kubectl rollout status deployment/fraud-detection

  deploy-production:
    needs: deploy-staging
    runs-on: ubuntu-latest
    environment: production
    steps:
      - name: Deploy to production (canary)
        run: |
          kubectl set image deployment/fraud-detection-canary \
            fraud-container=${{ env.REGISTRY }}/fraud-model:${{ github.sha }}
      - name: Monitor canary
        run: |
          ./monitor_canary.sh --duration=5m --threshold=0.95
      - name: Promote to full production
        run: |
          kubectl set image deployment/fraud-detection \
            fraud-container=${{ env.REGISTRY }}/fraud-model:${{ github.sha }}
```

4. Monitoring, Logging, and Performance Optimization

Responsibilities:

- Implement comprehensive monitoring for model performance metrics
- Set up alerting for model drift and data drift detection
- Configure logging for all predictions and inference requests
- Monitor system resources (CPU, memory, network)
- Track model accuracy, precision, recall in production
- Implement retry and circuit breaker patterns
- Optimize inference performance (caching, batching, parallelization)
- Establish SLOs (Service Level Objectives) and SLAs

Monitoring Implementation:

python

```
# Monitoring and alerting setup
from prometheus_client import Counter, Histogram, Gauge
```

```python
import logging

# Metrics
prediction_counter = Counter(
    'fraud_predictions_total',
    'Total number of predictions',
    ['model_version', 'prediction_result']
)

prediction_latency = Histogram(
    'fraud_prediction_duration_seconds',
    'Prediction latency',
    buckets=[0.01, 0.05, 0.1, 0.5, 1.0]
)

model_drift_score = Gauge(
    'fraud_model_drift_score',
    'Model drift detection score'
)

# Monitoring checks
def monitor_model_performance():
    # Check for data drift
    data_drift = calculate_drift_score(
        current_data,
        training_data_reference
    )

    # Check for concept drift
    concept_drift = evaluate_model_performance(
        model,
        recent_data_with_labels
    )

    # Alert if drift detected
    if data_drift > DRIFT_THRESHOLD:
        send_alert("Data drift detected in fraud model")

    if concept_drift > CONCEPT_DRIFT_THRESHOLD:
```

```
            send_alert("Model performance degradation detected")
            trigger_model_retraining()

    # Performance optimization example
    class OptimizedFraudDetector:
        def __init__(self):
            self.cache = {}
            self.batch_size = 100

        def predict_batch(self, transactions):
            """Batch prediction for better throughput"""
            # Batch preprocessing
            features = [self.prepare_features(tx) for tx in transactions]

            # Use batch prediction
            predictions = self.model.predict(features)

            # Cache frequent predictions
            for tx, pred in zip(transactions, predictions):
                cache_key = self.generate_cache_key(tx)
                self.cache[cache_key] = pred

            return predictions

        def predict_with_timeout(self, transaction, timeout=0.5):
            """Predict with timeout for SLA compliance"""
            return asyncio.wait_for(
                self.predict(transaction),
                timeout=timeout
            )
```

Additional Responsibilities (Beyond Four):

- **Security Implementation:** API authentication/authorization, data encryption, access controls
- **Documentation:** API documentation, deployment procedures, operational runbooks
- **Stakeholder Communication:** Updates on deployment progress, performance metrics
- **Disaster Recovery:** Backup strategies, failover mechanisms, data replication
- **Cost Optimization:** Monitor cloud costs, optimize resource utilization

---

BONUS (5 marks)

What is the difference between Structured, Semi-structured, and Unstructured Data?

1. Structured Data

Definition: Data that is organized in a highly formatted, tabular structure with a fixed schema and well-defined relationships.

Characteristics:

- **Predefined Schema:** Columns and data types are fixed
- **Relational Structure:** Data organized in rows and columns
- **Strong Data Integrity:** ACID compliance, referential integrity
- **Easy to Query:** SQL and other structured query languages
- **Machine-Readable:** Can be processed directly by programs
- **Metadata-Driven:** Schema defines the data structure

Examples:

- Relational databases (MySQL, PostgreSQL, Oracle, SQL Server)
- CSV files with consistent structure
- Excel spreadsheets with proper formatting
- Data warehouses and data marts

Storage & Access:

- Stored in tables with foreign key relationships
- Accessed using SQL and ORMs
- Indexed for fast retrieval

Pros & Cons:

- ✓ Easy to search, analyze, and aggregate
- ✓ Strong data integrity and consistency
- ✓ Established tooling and best practices
- ✗ Rigid schema makes changes difficult
- ✗ Not suitable for high-velocity, unstructured data
- ✗ Storage is often more expensive

2. Semi-structured Data

Definition: Data that doesn't conform to a rigid tabular structure but contains tags or markers to separate semantic elements and enforce hierarchies.

Characteristics:

- **Flexible Schema:** Does not follow a fixed structure
- **Self-Describing:** Contains metadata within the data
- **Hierarchical:** Can represent parent-child relationships
- **Nested Structure:** Supports multiple nesting levels
- **Evolving:** Schema can change over time
- **Machine-Readable:** Can be processed programmatically

Examples:

- JSON (JavaScript Object Notation)
- XML (Extensible Markup Language)
- YAML (YAML Ain't Markup Language)
- Avro (binary format with JSON schema)
- Parquet (columnar storage with schema)
- NoSQL databases (MongoDB, CouchDB, Cassandra)

Storage & Access:

- Stored as key-value pairs, documents, or graph structures
- Accessed using NoSQL queries or specialized parsers
- Supports both schema-on-read and schema-on-write

Pros & Cons:

- ✓ Flexible and adaptable to changing requirements
- ✓ Supports complex nested structures
- ✓ Easier integration with modern applications
- ✗ More complex to query and analyze
- ✗ Less efficient for ad-hoc reporting
- ✗ May require specialized query languages

3. Unstructured Data

Definition: Data that lacks any predefined format or organization, making it difficult to process using traditional database methods.

Characteristics:

- **No Predefined Format:** Not organized in a logical manner
- **Human-Generated:** Primarily created by humans
- **Rich Content:** Contains text, images, video, audio

- **Context-Dependent:** Meaning derived from context
- **Volume:** Often comprises >80% of enterprise data
- **Challenge:** Hard to search, analyze, and extract value

Examples:

- Text documents (PDFs, Word, PowerPoint, plain text)
- Images (JPEG, PNG, GIF, TIFF, RAW)
- Video files (MP4, AVI, MOV, WMV)
- Audio files (MP3, WAV, AAC, FLAC)
- Social media content (tweets, comments, posts)
- Emails and chat messages
- Web pages and HTML content
- Sensor data (IoT, machine logs, telemetry)
- Satellite imagery and medical scans

Storage & Access:

- Stored as files in data lakes (S3, HDFS, Azure Blob)
- Accessed using specialized tools and APIs
- Requires metadata for organization and search

Pros & Cons:

- ✓ Contains rich, valuable information
- ✓ Important for AI/ML applications
- ✓ Can provide competitive advantage
- ✗ Hard to analyze without NLP/computer vision
- ✗ Storage and processing require significant resources
- ✗ Compliance and governance are challenging

Comparison Table:

Feature	Structured	Semi-structured	Unstructured
Schema	Fixed, predefined	Flexible, evolving	None
Data Model	Relational (2D tables)	Hierarchical/Graph	Any
Data Integrity	High (ACID)	Medium	Low

Feature	Structured	Semi-structured	Unstructured
Query Language	SQL	NoSQL, custom	Specialized
Searchability	Easy	Moderate	Difficult
Analysis	Business Intelligence	Web/NoSQL apps	AI/ML, NLP
Storage	DW, Relational DB	NoSQL DB, Data Lake	Data Lake, File Systems
Volume	Small (<10%)	Medium (10-20%)	Large (>80%)
Speed	Fast	Moderate	Slow
Cost	Higher	Medium	Lower (storage)
Examples	Banking transactions	JSON API responses	Video surveillance

Modern Data Management Trends:

1. **Data Lakes:** Store all data types together (structured, semi-structured, unstructured)
2. **Schema-on-Read:** Apply structure when reading, not when writing
3. **Multi-Model Databases:** Support multiple data models (document, graph, key-value)
4. **Unified Analytics:** Combine structured and unstructured data for better insights
5. **AI/ML Integration:** Extract value from unstructured data using AI

Real-World Example in Banking:

```
text
Structured: Customer account details in SQL database
  → Account #, Balance, CustomerID, AccountType

Semi-structured: Transaction metadata in JSON
  → {"transaction_id": "T123", "amount": 5000,
     "location": {"lat": -1.286, "lng": 36.817},
     "merchant": {"id": "M789", "category": "retail"}}
```

Unstructured: Customer support emails and chat logs
  → "I want to report fraud in my account number 12345..."
     [Attachment: Screenshot_of_transaction.jpg]
     Voice recording of customer call

END OF ASSESSMENT